

Praktikum 11 - File Systems

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

TUJUAN PRAKTIKUM

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep file dan mengimplementasikan operasi file system dalam C
 2. Mengimplementasikan berbagai metode akses file (sequential, direct, indexed)
 3. Mengelola struktur direktori, permission, dan protection mechanism
 4. Menggunakan memory-mapped files untuk akses data yang efisien
 5. Memahami proses mounting dan file sharing di Linux
 6. Membuat simulasi allocation methods (contiguous, linked, indexed)
-

PERSIAPAN PRAKTIKUM

Tools yang Dibutuhkan:

```
# Update sistem
sudo apt update

# Install tools yang diperlukan
sudo apt install build-essential gcc vim tree htop -y

# Verifikasi GCC
gcc --version
```

Setup Direktori Kerja:

```
# Buat direktori praktikum
mkdir -p ~/praktikum_filesystem
cd ~/praktikum_filesystem

# Buat subdirectory
mkdir -p {source,build,data,mount_test}
```

BAGIAN 1: FILE-SYSTEM INTERFACE

1.1 File Concept dan Operasi Dasar

Praktikum 1.1: Eksplorasi Atribut File

```
# Buat file test
echo "Praktikum File System - UM Jember" > test_file.txt

# Lihat atribut lengkap
stat test_file.txt

# Output yang perlu diperhatikan:
# - File size (ukuran)
# - Inode number
# - Access, Modify, Change time
# - Permissions (mode)
```

Tugas Analisis:

Atribut	Keterangan	Nilai
Inode	Nomor unik file	...
Size	Ukuran dalam byte	...
Blocks	Jumlah blok yang dialokasikan	...
Access Time	Waktu terakhir diakses	...
Modify Time	Waktu terakhir dimodifikasi	...

Praktikum 1.2: Implementasi File Operations dalam C

Buat file `file_operations.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>

// Struktur untuk menyimpan metadata file
typedef struct {
    char name[256];
    long size;
    time_t created;
    time_t modified;
    mode_t permissions;
} FileMetadata;
```

```
// Fungsi untuk membuat file
int create_file(const char *filename) {
    FILE *fp = fopen(filename, "w");
    if (fp == NULL) {
        perror("Error creating file");
        return -1;
    }
    printf("[CREATE] File '%s' berhasil dibuat\n", filename);
    fclose(fp);
    return 0;
}

// Fungsi untuk menulis file
int write_file(const char *filename, const char *data) {
    FILE *fp = fopen(filename, "a");
    if (fp == NULL) {
        perror("Error opening file for write");
        return -1;
    }
    fprintf(fp, "%s\n", data);
    printf("[WRITE] Data berhasil ditulis ke '%s'\n", filename);
    fclose(fp);
    return 0;
}

// Fungsi untuk membaca file
int read_file(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        perror("Error opening file for read");
        return -1;
    }

    printf("\n[READ] Isi file '%s':\n", filename);
    printf("=====\n");

    char buffer[256];
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }

    printf("=====\n\n");
    fclose(fp);
    return 0;
}

// Fungsi untuk mendapatkan metadata file
int get_file_metadata(const char *filename, FileMetadata *meta) {
    struct stat st;

    if (stat(filename, &st) != 0) {
        perror("Error getting file metadata");
        return -1;
    }
}
```

```

    }

    strncpy(meta->name, filename, sizeof(meta->name) - 1);
    meta->size = st.st_size;
    meta->created = st.st_ctime;
    meta->modified = st.st_mtime;
    meta->permissions = st.st_mode;

    return 0;
}

// Fungsi untuk menampilkan metadata
void display_metadata(FileMetadata *meta) {
    printf("\n[METADATA] Informasi File:\n");
    printf("=====\n");
    printf("Nama File      : %s\n", meta->name);
    printf("Ukuran        : %ld bytes\n", meta->size);
    printf("Dibuat         : %s", ctime(&meta->created));
    printf("Dimodifikasi  : %s", ctime(&meta->modified));
    printf("Permissions   : %o\n", meta->permissions & 0777);
    printf("=====\n\n");
}

// Fungsi untuk menghapus file
int delete_file(const char *filename) {
    if (remove(filename) == 0) {
        printf("[DELETE] File '%s' berhasil dihapus\n", filename);
        return 0;
    } else {
        perror("Error deleting file");
        return -1;
    }
}

// Fungsi untuk rename file
int rename_file(const char *oldname, const char *newname) {
    if (rename(oldname, newname) == 0) {
        printf("[RENAME] File '%s' -> '%s'\n", oldname, newname);
        return 0;
    } else {
        perror("Error renaming file");
        return -1;
    }
}

int main() {
    printf("=== PRAKTIKUM FILE OPERATIONS ===\n\n");

    const char *filename = "mahasiswa.txt";
    FileMetadata meta;

    // 1. CREATE
    create_file(filename);

```

```
// 2. WRITE
write_file(filename, "NIM: 202310001");
write_file(filename, "Nama: Ahmad Fauzi");
write_file(filename, "Prodi: Teknik Informatika");
write_file(filename, "Universitas: UM Jember");

// 3. READ
read_file(filename);

// 4. GET METADATA
get_file_metadata(filename, &meta);
display_metadata(&meta);

// 5. RENAME
rename_file(filename, "data_mahasiswa.txt");

// 6. READ lagi setelah rename
read_file("data_mahasiswa.txt");

// 7. DELETE (uncomment untuk menghapus)
// delete_file("data_mahasiswa.txt");

printf("Program selesai.\n");
return 0;
}
```

Compile dan Jalankan:

```
cd ~/praktikum_filesystem/source
gcc file_operations.c -o ../build/file_operations
../build/file_operations
```

Tugas:

1. Jalankan program dan dokumentasikan output
2. Tambahkan fungsi `append_file()` untuk menambah data tanpa overwrite
3. Tambahkan fungsi `copy_file()` untuk menyalin file

1.3 Access Methods

Praktikum 1.3: Sequential vs Direct Access

Buat file `access_methods.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

```

#define RECORD_SIZE 50

// Struktur untuk record mahasiswa
typedef struct {
    int nim;
    char nama[30];
    float ipk;
} Mahasiswa;

// SEQUENTIAL ACCESS: Baca semua record dari awal
void sequential_access(const char *filename) {
    FILE *fp = fopen(filename, "rb");
    if (!fp) {
        perror("Error opening file");
        return;
    }

    printf("\n=== SEQUENTIAL ACCESS ===\n");
    printf("%-10s %-20s %-10s\n", "NIM", "Nama", "IPK");
    printf("-----\n");

    Mahasiswa mhs;
    clock_t start = clock();
    int count = 0;

    while (fread(&mhs, sizeof(Mahasiswa), 1, fp) == 1) {
        printf("%-10d %-20s %-10.2f\n", mhs.nim, mhs.nama, mhs.ipk);
        count++;
    }

    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nTotal record: %d\n", count);
    printf("Waktu akses: %f detik\n", time_taken);

    fclose(fp);
}

// DIRECT ACCESS: Langsung ke record tertentu
void direct_access(const char *filename, int record_number) {
    FILE *fp = fopen(filename, "rb");
    if (!fp) {
        perror("Error opening file");
        return;
    }

    printf("\n=== DIRECT ACCESS (Record #%d) ===\n", record_number);

    clock_t start = clock();

    // Hitung offset
    long offset = (record_number - 1) * sizeof(Mahasiswa);

```

```

// Seek ke posisi
if (fseek(fp, offset, SEEK_SET) != 0) {
    printf("Error: Record tidak ditemukan\n");
    fclose(fp);
    return;
}

Mahasiswa mhs;
if (fread(&mhs, sizeof(Mahasiswa), 1, fp) == 1) {
    printf("%-10s %-20s %-10s\n", "NIM", "Nama", "IPK");
    printf("-----\n");
    printf("%-10d %-20s %-10.2f\n", mhs.nim, mhs.nama, mhs.ipk);
} else {
    printf("Error membaca record\n");
}

clock_t end = clock();
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nWaktu akses: %f detik\n", time_taken);

fclose(fp);
}

// Fungsi untuk membuat file database
void create_database(const char *filename, int num_records) {
    FILE *fp = fopen(filename, "wb");
    if (!fp) {
        perror("Error creating database");
        return;
    }

    printf("Membuat database dengan %d record...\n", num_records);

    Mahasiswa mhs;
    for (int i = 1; i <= num_records; i++) {
        mhs.nim = 202310000 + i;
        sprintf(mhs.nama, "Mahasiswa_%d", i);
        mhs.ipk = 2.5 + (rand() % 150) / 100.0; // IPK random 2.5-4.0

        fwrite(&mhs, sizeof(Mahasiswa), 1, fp);
    }

    printf("Database berhasil dibuat!\n");
    fclose(fp);
}

int main() {
    const char *db_file = "../data/mahasiswa.dat";

    printf("=== PRAKTIKUM ACCESS METHODS ===\n\n");

    // Buat database dengan 100 record
    create_database(db_file, 100);

```

```
// Test Sequential Access
sequential_access(db_file);

// Test Direct Access – record ke-50
direct_access(db_file, 50);

// Test Direct Access – record ke-1
direct_access(db_file, 1);

// Test Direct Access – record ke-100
direct_access(db_file, 100);

printf("\n=== PERBANDINGAN ===\n");
printf("Sequential Access: Cocok untuk membaca semua data\n");
printf("Direct Access: Cocok untuk akses data spesifik\n");

return 0;
}
```

Compile dan Jalankan:

```
gcc access_methods.c -o ../build/access_methods
../build/access_methods
```

Analisis:

1. Bandingkan waktu akses sequential vs direct
2. Kapan menggunakan sequential access?
3. Kapan menggunakan direct access?

1.4 Directory Structure dan Protection

Praktikum 1.4: Membuat Tree Directory Structure

```
cd ~/praktikum_filesystem

# Buat struktur tree-structured directory
mkdir -p
filesystem_tree/{root,users/{user1,user2,user3}/{documents,downloads,pictures},shared,system/{bin,lib,etc}}

# Navigasi dan buat file
cd filesystem_tree

# User directories
echo "Document user1" > users/user1/documents/file1.txt
echo "Document user2" > users/user2/documents/file2.txt
```



```
echo "Shared data" > shared/shared_doc.txt
echo "System config" > system/etc/config.txt

# Visualisasi struktur
tree .
```

Output yang diharapkan:

```
.
├── root
├── shared
│   └── shared_doc.txt
├── system
│   ├── bin
│   ├── etc
│   │   └── config.txt
│   └── lib
└── users
    ├── user1
    │   ├── documents
    │   │   └── file1.txt
    │   ├── downloads
    │   └── pictures
    ├── user2
    │   ├── documents
    │   │   └── file2.txt
    │   ├── downloads
    │   └── pictures
    └── user3
        ├── documents
        ├── downloads
        └── pictures
```

Praktikum 1.5: File Protection dan Permissions

```
cd ~/praktikum_filesystem

# Buat file untuk testing
echo "Public file - Everyone can read" > public.txt
echo "Private file - Only owner" > private.txt
echo "Group file - Owner and group" > group.txt
echo "#!/bin/bash\nnecho 'Executable script'" > script.sh

# Set different permissions
chmod 644 public.txt    # rw-r--r--
chmod 600 private.txt   # rw-----
chmod 660 group.txt     # rw-rw----
chmod 755 script.sh     # rwxr-xr-x
```

```
# Lihat permissions
ls -l

# Detail masing-masing file
stat public.txt | grep Access
stat private.txt | grep Access
stat group.txt | grep Access
stat script.sh | grep Access
```

Implementasi Permission Checker dalam C:

Buat file `permission_checker.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>

void check_permissions(const char *filename) {
    struct stat st;

    if (stat(filename, &st) != 0) {
        perror("Error getting file status");
        return;
    }

    printf("\n=== FILE PERMISSIONS: %s ===\n", filename);

    // Owner permissions
    printf("Owner: ");
    printf((st.st_mode & S_IRUSR) ? "r" : "-");
    printf((st.st_mode & S_IWUSR) ? "w" : "-");
    printf((st.st_mode & S_IXUSR) ? "x" : "-");

    // Group permissions
    printf(" | Group: ");
    printf((st.st_mode & S_IRGRP) ? "r" : "-");
    printf((st.st_mode & S_IWGRP) ? "w" : "-");
    printf((st.st_mode & S_IXGRP) ? "x" : "-");

    // Others permissions
    printf(" | Others: ");
    printf((st.st_mode & S_IROTH) ? "r" : "-");
    printf((st.st_mode & S_IWOTH) ? "w" : "-");
    printf((st.st_mode & S_IXOTH) ? "x" : "-");

    printf("\nOctal: %o\n", st.st_mode & 0777);

    // Check access untuk current user
    printf("\nAccess untuk current user:\n");
```

```

    printf("  Read:    %s\n", (access(filename, R_OK) == 0) ? "YES" :
"NO");
    printf("  Write:   %s\n", (access(filename, W_OK) == 0) ? "YES" :
"NO");
    printf("  Execute: %s\n", (access(filename, X_OK) == 0) ? "YES" :
"NO");
}

int change_permissions(const char *filename, mode_t mode) {
    if (chmod(filename, mode) != 0) {
        perror("Error changing permissions");
        return -1;
    }
    printf("\n[CHMOD] Permissions changed to %o\n", mode);
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    const char *filename = argv[1];

    printf("=== PERMISSION CHECKER ===\n");

    // Check current permissions
    check_permissions(filename);

    // Contoh: ubah ke 755
    if (argc > 2 && strcmp(argv[2], "change") == 0) {
        change_permissions(filename, 0755);
        check_permissions(filename);
    }

    return 0;
}

```

Compile dan Test:

```

gcc permission_checker.c -o ../build/permission_checker

# Test dengan berbagai file
../build/permission_checker public.txt
../build/permission_checker private.txt
../build/permission_checker script.sh

# Test change permission
../build/permission_checker public.txt change

```

1.6 Memory-Mapped Files

Praktikum 1.6: Implementasi Memory-Mapped I/O

Buat file `mmap_demo.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <time.h>

#define FILE_SIZE 1024

// Fungsi untuk membuat file test
void create_test_file(const char *filename) {
    FILE *fp = fopen(filename, "w");
    if (!fp) {
        perror("Error creating file");
        return;
    }

    // Tulis data awal
    for (int i = 0; i < 20; i++) {
        fprintf(fp, "Baris %d: Data untuk testing memory-mapped I/O\n", i
+ 1);
    }

    fclose(fp);
    printf("Test file created: %s\n", filename);
}

// Traditional I/O (fread/fwrite)
double traditional_io(const char *filename) {
    clock_t start = clock();

    FILE *fp = fopen(filename, "r+");
    if (!fp) {
        perror("Error opening file");
        return -1;
    }

    char buffer[FILE_SIZE];
    size_t bytes_read = fread(buffer, 1, FILE_SIZE, fp);

    // Modifikasi data
    if (bytes_read > 0 && bytes_read < FILE_SIZE) {
        buffer[0] = 'X';
    }
}
```

```

    }

    // Tulis kembali
    fseek(fp, 0, SEEK_SET);
    fwrite(buffer, 1, bytes_read, fp);

    fclose(fp);

    clock_t end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

// Memory-Mapped I/O
double mmap_io(const char *filename) {
    clock_t start = clock();

    int fd = open(filename, O_RDWR);
    if (fd == -1) {
        perror("Error opening file");
        return -1;
    }

    struct stat sb;
    fstat(fd, &sb);

    // Map file ke memory
    char *map = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) {
        perror("Error mapping file");
        close(fd);
        return -1;
    }

    // Modifikasi langsung di memory
    map[0] = 'M';

    // Sync ke disk
    msync(map, sb.st_size, MS_SYNC);

    // Cleanup
    munmap(map, sb.st_size);
    close(fd);

    clock_t end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

// Demonstrasi sharing antar process
void mmap_sharing_demo(const char *filename) {
    int fd = open(filename, O_RDWR | O_CREAT, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }

```

```

    }

    // Set ukuran file
    ftruncate(fd, FILE_SIZE);

    // Map file
    char *map = mmap(NULL, FILE_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) {
        perror("Error mapping");
        close(fd);
        return;
    }

    printf("\n=== MEMORY-MAPPED FILE SHARING ===\n");

    // Tulis data
    sprintf(map, "Data dari process %d", getpid());
    printf("Written: %s\n", map);

    // Sync
    msync(map, FILE_SIZE, MS_SYNC);

    printf("File di-map ke address: %p\n", (void*)map);
    printf("File bisa diakses oleh process lain yang mem-map file yang sama\n");

    munmap(map, FILE_SIZE);
    close(fd);
}

int main() {
    printf("=== PRAKTIKUM MEMORY-MAPPED FILES ===\n\n");

    const char *test_file1 = "../data/test_traditional.txt";
    const char *test_file2 = "../data/test_mmap.txt";
    const char *shared_file = "../data/shared_mmap.txt";

    // Buat test files
    create_test_file(test_file1);
    create_test_file(test_file2);

    printf("\n=== PERFORMANCE COMPARISON ===\n");

    // Test Traditional I/O
    double time_traditional = traditional_io(test_file1);
    printf("Traditional I/O time: %.6f seconds\n", time_traditional);

    // Test Memory-Mapped I/O
    double time_mmap = mmap_io(test_file2);
    printf("Memory-Mapped I/O time: %.6f seconds\n", time_mmap);

    // Perbandingan
    if (time_mmap < time_traditional) {

```

```

        printf("\nMemory-Mapped I/O lebih cepat %.2fx\n",
               time_traditional / time_mmap);
    }

    // Demo sharing
    mmap_sharing_demo(shared_file);

    printf("\n=== KEUNTUNGAN MEMORY-MAPPED FILES ===\n");
    printf("1. Akses lebih cepat (langsung ke memory)\n");
    printf("2. Efisien untuk file besar\n");
    printf("3. Mudah sharing antar process\n");
    printf("4. Tidak perlu buffer tambahan\n");

    return 0;
}

```

Compile dan Jalankan:

```

gcc mmap_demo.c -o ../build/mmap_demo
../build/mmap_demo

```

BAGIAN 2: FILE-SYSTEM IMPLEMENTATION

2.1 Simulasi Allocation Methods

Praktikum 2.1: Contiguous Allocation

Buat file `contiguous_allocation.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DISK_SIZE 100
#define MAX_FILES 20

typedef struct {
    char name[50];
    int start_block;
    int length;
    int allocated;
} File;

typedef struct {
    int blocks[DISK_SIZE];
    File files[MAX_FILES];
    int file_count;
} ContiguousDisk;

```

```

void init_disk(ContiguousDisk *disk) {
    for (int i = 0; i < DISK_SIZE; i++) {
        disk->blocks[i] = -1; // -1 = free
    }
    disk->file_count = 0;
    printf("Disk initialized: %d blocks\n", DISK_SIZE);
}

int allocate_file(ContiguousDisk *disk, const char *name, int size) {
    // Cari contiguous free space
    int start = -1;
    int count = 0;

    for (int i = 0; i < DISK_SIZE; i++) {
        if (disk->blocks[i] == -1) {
            if (start == -1) start = i;
            count++;

            if (count == size) {
                // Alokasi file
                int file_id = disk->file_count;

                for (int j = start; j < start + size; j++) {
                    disk->blocks[j] = file_id;
                }

                strcpy(disk->files[file_id].name, name);
                disk->files[file_id].start_block = start;
                disk->files[file_id].length = size;
                disk->files[file_id].allocated = 1;
                disk->file_count++;

                printf("[OK] '%s' allocated at block %d-%d (%d blocks)\n",
                    name, start, start + size - 1, size);
                return 0;
            }
        } else {
            start = -1;
            count = 0;
        }
    }

    printf("[FAIL] Not enough contiguous space for '%s'\n", name);
    return -1;
}

int deallocate_file(ContiguousDisk *disk, const char *name) {
    for (int i = 0; i < disk->file_count; i++) {
        if (strcmp(disk->files[i].name, name) == 0 && disk->files[i].allocated) {
            int start = disk->files[i].start_block;
            int len = disk->files[i].length;

```



```

        for (int j = start; j < start + len; j++) {
            disk->blocks[j] = -1;
        }

        disk->files[i].allocated = 0;
        printf("[OK] '%s' deallocated\n", name);
        return 0;
    }
}

printf("[FAIL] File '%s' not found\n", name);
return -1;
}

void display_disk(ContiguousDisk *disk) {
    printf("\n===== DISK LAYOUT =====\n");

    for (int i = 0; i < DISK_SIZE; i += 10) {
        printf("Block %3d-%3d: ", i, i + 9);
        for (int j = i; j < i + 10 && j < DISK_SIZE; j++) {
            if (disk->blocks[j] == -1) {
                printf("[ FREE ] ");
            } else {
                printf("[F%-2d:%02d] ", disk->blocks[j], j);
            }
        }
        printf("\n");
    }

    printf("\n===== FILE TABLE =====\n");
    printf("%-15s %-10s %-10s\n", "Filename", "Start", "Length");
    printf("-----\n");

    for (int i = 0; i < disk->file_count; i++) {
        if (disk->files[i].allocated) {
            printf("%-15s %-10d %-10d\n",
                disk->files[i].name,
                disk->files[i].start_block,
                disk->files[i].length);
        }
    }
    printf("=====\n\n");
}

void analyze_fragmentation(ContiguousDisk *disk) {
    int free_blocks = 0;
    int fragments = 0;
    int in_free_zone = 0;

    for (int i = 0; i < DISK_SIZE; i++) {
        if (disk->blocks[i] == -1) {
            free_blocks++;
            if (!in_free_zone) {
                fragments++;
            }
        }
    }
}

```

```

        in_free_zone = 1;
    }
} else {
    in_free_zone = 0;
}
}

printf("=== FRAGMENTATION ANALYSIS ===\n");
printf("Free blocks: %d\n", free_blocks);
printf("Free fragments: %d\n", fragments);
printf("Fragmentation level: %s\n",
        fragments > 5 ? "HIGH" : fragments > 2 ? "MEDIUM" : "LOW");
printf("=====\n\n");
}

int main() {
    ContiguousDisk disk;

    printf("=== CONTIGUOUS ALLOCATION SIMULATION ===\n\n");

    init_disk(&disk);

    // Alokasi files
    allocate_file(&disk, "file1.txt", 5);
    allocate_file(&disk, "file2.txt", 10);
    allocate_file(&disk, "file3.txt", 7);
    allocate_file(&disk, "file4.txt", 8);

    display_disk(&disk);

    // Hapus file tengah (simulasi fragmentation)
    deallocate_file(&disk, "file2.txt");

    display_disk(&disk);
    analyze_fragmentation(&disk);

    // Coba alokasi file besar (gagal karena fragmentation)
    allocate_file(&disk, "file5.txt", 12);

    // Alokasi file kecil (berhasil)
    allocate_file(&disk, "file6.txt", 5);

    display_disk(&disk);
    analyze_fragmentation(&disk);

    printf("\n=== KESIMPULAN ===\n");
    printf("- Contiguous allocation: akses cepat O(1)\n");
    printf("- Kekurangan: external fragmentation\n");
    printf("- File tidak bisa grow dinamis\n");

    return 0;
}

```

Compile dan Jalankan:

```
gcc contiguous_allocation.c -o ../build/contiguous_allocation
../build/contiguous_allocation
```

Praktikum 2.2: Linked Allocation

Buat file `linked_allocation.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DISK_SIZE 100
#define MAX_FILES 20

typedef struct Block {
    int file_id;
    int next;
    char data[10];
} Block;

typedef struct {
    char name[50];
    int start_block;
    int size;
    int allocated;
} LinkedFile;

typedef struct {
    Block blocks[DISK_SIZE];
    LinkedFile files[MAX_FILES];
    int file_count;
} LinkedDisk;

void init_disk(LinkedDisk *disk) {
    for (int i = 0; i < DISK_SIZE; i++) {
        disk->blocks[i].file_id = -1;
        disk->blocks[i].next = -1;
        sprintf(disk->blocks[i].data, "B%d", i);
    }
    disk->file_count = 0;
    printf("Linked Disk initialized: %d blocks\n", DISK_SIZE);
}

int find_free_block(LinkedDisk *disk) {
    for (int i = 0; i < DISK_SIZE; i++) {
        if (disk->blocks[i].file_id == -1) {
            return i;
        }
    }
}
```

```

    }
}
return -1;
}

int allocate_file(LinkedDisk *disk, const char *name, int size) {
    int file_id = disk->file_count;
    int prev_block = -1;
    int first_block = -1;

    for (int i = 0; i < size; i++) {
        int block_num = find_free_block(disk);

        if (block_num == -1) {
            printf("[FAIL] Not enough space for '%s'\n", name);
            return -1;
        }

        if (first_block == -1) {
            first_block = block_num;
        }

        disk->blocks[block_num].file_id = file_id;

        if (prev_block != -1) {
            disk->blocks[prev_block].next = block_num;
        }

        prev_block = block_num;
    }

    // Last block points to -1
    disk->blocks[prev_block].next = -1;

    strcpy(disk->files[file_id].name, name);
    disk->files[file_id].start_block = first_block;
    disk->files[file_id].size = size;
    disk->files[file_id].allocated = 1;
    disk->file_count++;

    printf("[OK] '%s' allocated (%d blocks, start: %d)\n", name, size,
first_block);
    return 0;
}

int deallocate_file(LinkedDisk *disk, const char *name) {
    for (int i = 0; i < disk->file_count; i++) {
        if (strcmp(disk->files[i].name, name) == 0 && disk-
>files[i].allocated) {
            int current = disk->files[i].start_block;

            while (current != -1) {
                int next = disk->blocks[current].next;
                disk->blocks[current].file_id = -1;
            }
        }
    }
}

```

```

        disk->blocks[current].next = -1;
        current = next;
    }

    disk->files[i].allocated = 0;
    printf("[OK] '%s' deallocated\n", name);
    return 0;
}

}

printf("[FAIL] File '%s' not found\n", name);
return -1;
}

void sequential_read(LinkedDisk *disk, const char *name) {
    for (int i = 0; i < disk->file_count; i++) {
        if (strcmp(disk->files[i].name, name) == 0 && disk->files[i].allocated) {
            printf("\n=== SEQUENTIAL READ: %s ===\n", name);

            int current = disk->files[i].start_block;
            int step = 0;

            while (current != -1) {
                step++;
                printf("Step %d: Block %d -> Next: %d\n",
                    step, current, disk->blocks[current].next);
                current = disk->blocks[current].next;
            }

            printf("Total steps: %d\n", step);
            return;
        }
    }

    printf("File '%s' not found\n", name);
}

void display_disk(LinkedDisk *disk) {
    printf("\n===== LINKED DISK LAYOUT =====\n");

    for (int i = 0; i < disk->file_count; i++) {
        if (disk->files[i].allocated) {
            printf("\nFile: %s (Start: %d, Size: %d)\n",
                disk->files[i].name,
                disk->files[i].start_block,
                disk->files[i].size);

            printf("  Chain: ");
            int current = disk->files[i].start_block;
            while (current != -1) {
                printf("%d", current);
                current = disk->blocks[current].next;
                if (current != -1) printf(" -> ");
            }
        }
    }
}

```

```

        }
        printf(" -> NULL\n");
    }
}

printf("\n===== \n\n");
}

int main() {
    LinkedDisk disk;

    printf("=== LINKED ALLOCATION SIMULATION ===\n\n");

    init_disk(&disk);

    // Alokasi files
    allocate_file(&disk, "fileA.txt", 5);
    allocate_file(&disk, "fileB.txt", 8);
    allocate_file(&disk, "fileC.txt", 4);

    display_disk(&disk);

    // Sequential read
    sequential_read(&disk, "fileB.txt");

    // Hapus file
    deallocate_file(&disk, "fileA.txt");

    display_disk(&disk);

    // Alokasi lagi (akan mengisi holes)
    allocate_file(&disk, "fileD.txt", 6);

    display_disk(&disk);

    printf("\n=== KESIMPULAN ===\n");
    printf("- Linked allocation: no external fragmentation\n");
    printf("- File bisa grow dinamis\n");
    printf("- Kekurangan: random access lambat O(n)\n");
    printf("- Pointer overhead\n");

    return 0;
}

```

Compile dan Jalankan:

```

gcc linked_allocation.c -o ../build/linked_allocation
../build/linked_allocation

```

2.2 Free-Space Management

Praktikum 2.3: Bit Vector Implementation

Buat file `free_space_bitmap.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DISK_SIZE 64

typedef struct {
    unsigned char bitmap[DISK_SIZE / 8]; // 1 byte = 8 bits
    int total_blocks;
    int free_blocks;
} BitmapDisk;

void init_bitmap(BitmapDisk *disk) {
    memset(disk->bitmap, 0, sizeof(disk->bitmap));
    disk->total_blocks = DISK_SIZE;
    disk->free_blocks = DISK_SIZE;
    printf("Bitmap initialized: %d blocks\n", DISK_SIZE);
}

int is_block_free(BitmapDisk *disk, int block_num) {
    int byte_index = block_num / 8;
    int bit_index = block_num % 8;
    return !(disk->bitmap[byte_index] & (1 << bit_index));
}

void set_block(BitmapDisk *disk, int block_num, int used) {
    int byte_index = block_num / 8;
    int bit_index = block_num % 8;

    if (used) {
        disk->bitmap[byte_index] |= (1 << bit_index);
        disk->free_blocks--;
    } else {
        disk->bitmap[byte_index] &= ~(1 << bit_index);
        disk->free_blocks++;
    }
}

int allocate_blocks(BitmapDisk *disk, int count) {
    int allocated = 0;
    int start_block = -1;

    printf("\nAllocating %d blocks...\n", count);

    for (int i = 0; i < DISK_SIZE && allocated < count; i++) {
        if (is_block_free(disk, i)) {
```

```

        if (start_block == -1) start_block = i;
        set_block(disk, i, 1);
        allocated++;
        printf("  Block %d allocated\n", i);
    }
}

if (allocated == count) {
    printf("[OK] %d blocks allocated starting at %d\n", count,
start_block);
    return start_block;
} else {
    printf("[FAIL] Only %d blocks available\n", allocated);
    return -1;
}
}

void free_blocks(BitmapDisk *disk, int start, int count) {
    printf("\nFreeing %d blocks from %d...\n", count, start);

    for (int i = start; i < start + count && i < DISK_SIZE; i++) {
        set_block(disk, i, 0);
        printf("  Block %d freed\n", i);
    }

    printf("[OK] Blocks freed\n");
}

void display_bitmap(BitmapDisk *disk) {
    printf("\n===== BITMAP DISPLAY =====\n");
    printf("Total blocks: %d | Free: %d | Used: %d\n",
        disk->total_blocks, disk->free_blocks,
        disk->total_blocks - disk->free_blocks);
    printf("\n");

    for (int i = 0; i < DISK_SIZE; i++) {
        if (i % 8 == 0) printf("Block %2d-%2d: ", i, i + 7);

        printf("%c ", is_block_free(disk, i) ? '0' : '1');

        if ((i + 1) % 8 == 0) printf("\n");
    }

    printf("=====\n\n");
}

void search_free_blocks(BitmapDisk *disk, int size) {
    printf("\nSearching for %d contiguous free blocks...\n", size);

    int count = 0;
    int start = -1;

    for (int i = 0; i < DISK_SIZE; i++) {
        if (is_block_free(disk, i)) {

```



```

        if (start == -1) start = i;
        count++;

        if (count == size) {
            printf("[FOUND] Blocks %d-%d are free\n", start, start +
size - 1);
            return;
        }
    } else {
        start = -1;
        count = 0;
    }
}

printf("[NOT FOUND] No %d contiguous blocks available\n", size);
}

int main() {
    BitmapDisk disk;

    printf("=== FREE-SPACE BITMAP SIMULATION ===\n\n");

    init_bitmap(&disk);
    display_bitmap(&disk);

    // Alokasi beberapa blocks
    allocate_blocks(&disk, 5);
    allocate_blocks(&disk, 8);
    allocate_blocks(&disk, 3);

    display_bitmap(&disk);

    // Free blocks
    free_blocks(&disk, 5, 8);

    display_bitmap(&disk);

    // Search contiguous space
    search_free_blocks(&disk, 10);
    search_free_blocks(&disk, 6);

    printf("\n=== KESIMPULAN ===\n");
    printf("- Bit Vector: simple & fast\n");
    printf("- 1 bit per block\n");
    printf("- Cocok untuk small disk\n");
    printf("- Wasted space untuk large disk\n");

    return 0;
}

```

Compile dan Jalankan:

```
gcc free_space_bitmap.c -o ../build/free_space_bitmap
../build/free_space_bitmap
```

BAGIAN 3: FILE-SYSTEM INTERNALS

3.1 File-System Mounting

Praktikum 3.1: Mount/Unmount Simulation

```
cd ~/praktikum_filesystem

# Buat direktori untuk simulasi
mkdir -p mount_test/{usb_drive,sd_card}
mkdir -p mount_points/{usb,sd}

# Buat file di "devices"
echo "Data di USB Drive" > mount_test/usb_drive/usb_data.txt
echo "Data di SD Card" > mount_test/sd_card/sd_data.txt

# Lihat struktur sebelum mount
echo "=== SEBELUM MOUNT ==="
tree mount_points/

# Simulasi mount menggunakan bind mount
sudo mount --bind mount_test/usb_drive mount_points/usb
sudo mount --bind mount_test/sd_card mount_points/sd

# Lihat struktur setelah mount
echo "=== SETELAH MOUNT ==="
tree mount_points/

# Lihat daftar mount
df -h | grep mount_points

# Akses file melalui mount point
cat mount_points/usb/usb_data.txt
cat mount_points/sd/sd_data.txt

# Unmount
sudo umount mount_points/usb
sudo umount mount_points/sd

echo "=== SETELAH UNMOUNT ==="
tree mount_points/
```

Praktikum 3.2: Mount Information

Buat file `mount_info.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <mntent.h>
#include <string.h>

void display_mount_info() {
    FILE *fp = setmntent("/proc/ mounts", "r");
    if (fp == NULL) {
        perror("Error opening /proc/ mounts");
        return;
    }

    printf("=== MOUNTED FILESYSTEMS ===\n");
    printf("%-20s %-20s %-10s %-30s\n",
        "Device", "Mount Point", "FS Type", "Options");
    printf("-----\n");

    struct mntent *entry;
    int count = 0;

    while ((entry = getmntent(fp)) != NULL) {
        // Filter hanya filesystem penting
        if (strstr(entry->mnt_type, "ext") != NULL ||
            strstr(entry->mnt_type, "ntfs") != NULL ||
            strstr(entry->mnt_type, "vfat") != NULL ||
            strstr(entry->mnt_type, "xfs") != NULL ||
            strcmp(entry->mnt_dir, "/") == 0) {

            printf("%-20s %-20s %-10s %-30s\n",
                entry->mnt_fsname,
                entry->mnt_dir,
                entry->mnt_type,
                entry->mnt_opts);

            count++;
        }
    }

    printf("\nTotal: %d filesystem mounted\n", count);

    endmntent(fp);
}

void check_mount_point(const char *path) {
    FILE *fp = setmntent("/proc/ mounts", "r");
    if (fp == NULL) {
        perror("Error opening /proc/ mounts");
        return;
    }

    struct mntent *entry;
```

```
int found = 0;

while ((entry = getmntent(fp)) != NULL) {
    if (strcmp(entry->mnt_dir, path) == 0) {
        printf("\n=== MOUNT POINT: %s ===\n", path);
        printf("Device: %s\n", entry->mnt_fsname);
        printf("Filesystem: %s\n", entry->mnt_type);
        printf("Options: %s\n", entry->mnt_opts);
        found = 1;
        break;
    }
}

if (!found) {
    printf("'%' is not a mount point\n", path);
}

endmntent(fp);
}

int main(int argc, char *argv[]) {
    printf("=== MOUNT INFORMATION TOOL ===\n\n");

    display_mount_info();

    if (argc > 1) {
        check_mount_point(argv[1]);
    } else {
        printf("\nUsage: %s <mount_point>\n", argv[0]);
        printf("Example: %s /home\n", argv[0]);
    }

    return 0;
}
```

Compile dan Jalankan:

```
gcc mount_info.c -o ../build/mount_info
../build/mount_info
../build/mount_info /home
```

3.3 File Sharing dan File Locking

Praktikum 3.3: File Locking Implementation

Buat file `file_locking.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/file.h>
#include <string.h>
#include <errno.h>

void demonstrate_advisory_locking(const char *filename) {
    printf("\n=== ADVISORY LOCKING DEMO ===\n");

    int fd = open(filename, O_RDWR | O_CREAT, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }

    printf("Trying to acquire EXCLUSIVE lock on '%s'...\n", filename);

    // Try to lock file
    if (flock(fd, LOCK_EX | LOCK_NB) == 0) {
        printf("[OK] Lock acquired!\n");
        printf("File is locked for 5 seconds...\n");

        // Tulis data
        const char *data = "Data written while file is locked\n";
        write(fd, data, strlen(data));

        sleep(5);

        // Release lock
        flock(fd, LOCK_UN);
        printf("[OK] Lock released\n");
    } else {
        if (errno == EWOULDBLOCK) {
            printf("[FAIL] File is locked by another process\n");
        } else {
            perror("Error locking file");
        }
    }

    close(fd);
}

void demonstrate_record_locking(const char *filename) {
    printf("\n=== RECORD LOCKING DEMO ===\n");

    int fd = open(filename, O_RDWR | O_CREAT, 0644);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }
}
```

```
struct flock lock;

// Lock bytes 0-9
lock.l_type = F_WRLCK;    // Write lock
lock.l_whence = SEEK_SET;
lock.l_start = 0;
lock.l_len = 10;          // Lock 10 bytes

printf("Trying to lock bytes 0-9...\n");

if (fcntl(fd, F_SETLK, &lock) == 0) {
    printf("[OK] Bytes 0-9 locked\n");
    printf("Writing to locked region...\n");

    lseek(fd, 0, SEEK_SET);
    write(fd, "LOCKED", 6);

    sleep(3);

    // Unlock
    lock.l_type = F_UNLCK;
    fcntl(fd, F_SETLK, &lock);
    printf("[OK] Lock released\n");
} else {
    perror("Failed to lock");
}

close(fd);
}

void check_lock_status(const char *filename) {
    printf("\n=== CHECK LOCK STATUS ===\n");

    int fd = open(filename, O_RDWR);
    if (fd == -1) {
        perror("Error opening file");
        return;
    }

    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0; // Check entire file

    if (fcntl(fd, F_GETLK, &lock) == -1) {
        perror("Error checking lock");
        close(fd);
        return;
    }

    if (lock.l_type == F_UNLCK) {
        printf("File is UNLOCKED\n");
    } else {
```

```

        printf("File is LOCKED\n");
        printf("  Lock type: %s\n",
            lock.l_type == F_RDLCK ? "READ" : "WRITE");
        printf("  PID: %d\n", lock.l_pid);
        printf("  Start: %ld\n", lock.l_start);
        printf("  Length: %ld\n", lock.l_len);
    }

    close(fd);
}

int main(int argc, char *argv[]) {
    const char *test_file = "../data/lock_test.txt";

    printf("=== FILE LOCKING DEMONSTRATION ===\n");

    // Demo 1: Advisory locking
    demonstrate_advisory_locking(test_file);

    // Demo 2: Record locking
    demonstrate_record_locking(test_file);

    // Check status
    check_lock_status(test_file);

    printf("\n=== KESIMPULAN ===\n");
    printf("- Advisory locking: process harus cooperate\n");
    printf("- Record locking: lock bagian file tertentu\n");
    printf("- Penting untuk file sharing\n");

    printf("\n[TIP] Jalankan program ini di 2 terminal berbeda\n");
    printf("        untuk melihat efek locking!\n");

    return 0;
}

```

Compile dan Jalankan:

```

gcc file_locking.c -o ../build/file_locking
../build/file_locking

```

Eksperimen: Jalankan di 2 terminal bersamaan untuk melihat efek locking!

REFERENSI

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.)
2. Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.)
3. Linux Manual Pages: [man 2 open](#), [man 2 mmap](#), [man 2 flock](#)
4. Materi kuliah Sistem Operasi - UM Jember 2026

